From pixels to a registered crystal database. A microscope frame comes in (via the app or batch.py, both calling pipeline.py:process_image) and the green channel is handed to SAM (flakeyield/sam_segment.py, the ViT-B checkpoint in models/), which proposes crystal-candidate masks with generous permissiveness — merged masks get seam-split, and truncated triangles get reconstructed — with results cached per frame so reprocessing is instant. Registration (experiments/sam_register.py) then figures out exactly where the GDS design sits on the image: it corner-matches each SAM detection to its designed twin in the x-flipped GDS layout, fits candidate "anchor" poses (tier 1: a single reliable triangle; tier 2: a joint fit over several visible crystals when no lone triangle is trustworthy), scores them by coverage and corner residual, then refines the winning pose with iterative closest point (ICP) — one global affine fitted over all mask-boundary correspondences — flagging the frame low-confidence if the fit is implausible (e.g. >8% anisotropic strain). With the design pinned to the image, experiments/build_crystal_db.py crops every GDS-outlined crystal from the original colour image and keeps only ones that pass two filters — not too bright relative to the frame background (drops the bright distractor crystals) and chromatically uniform after median-smoothing (drops multi-thickness junk while keeping wrinkled-but-single-colour flakes).

Wrinkle detection is a separately trained, point-supervised density U-Net, now run at native resolution (wrinkle_native.py, trained by train_wrinkle_native.py on the ~150 crops where you clicked individual wrinkles in wrinkle_points.json; the deployed checkpoint is out/wrinkle_native_prod.pt). The earlier model letterboxed every crop to 128×128 before inference — a ~10× downscale that <u>smeared</u> the thin 3–8 px wrinkle lines into nothing — so it was retired in favour of native-resolution tiling. Each click still becomes a Gaussian bump ($\sigma \approx 7$ px at native scale) in a target density map, and the network regresses that map from the contrast-normalized green-channel crop (2–98% stretch computed inside the crystal mask) under a masked MSE loss, so background outside the polygon is ignored. Training samples random 256 px native patches rather than one squashed frame; inference (wrinkle_native.py:predict_heatmap_native) tiles the whole crop with an overlapping 256 px window (128 px stride), Hann-blends the predictions, and masks to the crystal — yielding a full-resolution density map. The count is read off that map by integral: pixels below a relative background floor (5% of the crop's own maximum) are zeroed to remove diffuse haze, and the remaining density is divided by one bump's volume — the calibrated readout, which stays robust when dense wrinkles merge (exactly where isotropic peak-counting fragments and mis-counts). Marker(s) come from ridge-instance extraction (native_ridge.py): each physical wrinkle is traced as one extended crest polyline and marked once at its densest point, so overlays show one mark per wrinkle rather than a bead-string of maxima. A decoupled disagree flag still routes uncertain crops to human review — it fires when the grouped isotropic-peak count and the integral diverge by more than max(2, 70% of the integral).